

CHICAGO VANGUARD

IT462 Big Data Systems

Big Data Crime Pattern Analysis
Using Apache Spark

Group Number: 6

Prepared by

Name	ID
Aljwharah Alhowidy	443201015
Nora Albyahi	443200479
Mariam Alahmed	443204628
Raghad aldajani	443200551

Supervised by

Dr. Afshan Jafri
Second Semester 1447
2026

Table of Contents

1.	Introduction	4
1.1	Background and Context	4
1.2	Problem Statement	4
1.3	Objectives.....	5
1.4	Scope and Limitations	5
2.	Data Selection and Understanding	6
2.1	Dataset Description	6
2.2	Structure and Schema	7
2.3	Initial Data Quality Observations	7
2.4	Ethical and Legal Considerations	8
3.	Data Preprocessing	8
3.1	Cleaning	8
3.2	Reduction	9
3.3	Transformation	9
3.4	Pipeline Summary.....	10
3.5	Preprocessing Pipeline Summary.....	11
4.	RDD Operations.....	11
4.1	RDD Design.....	11
4.2	Key Transformations and Actions	11
4.3	Insights.....	12
4.4	Summary of RDD Findings.....	13
5.	SQL Operations	13
5.1	Temp View Creation	13
5.2	Key Analytical Queries	13
5.3	SQL Insights.....	14
5.4	Summary of SQL Findings	15
.6.....	Machine Learning Operations	16
6.1	Problem Formulation	16
6.2	Feature Pipeline and Preprocessing	16
6.3	Model Training and Evaluation.....	16

6.4	Cluster Analysis and Interpretation.....	18
6.5	Limitations and Future Improvements.....	19
.7	Conclusion	19
.8	References	19

Table of Figures

Figure 1 Crime Analysis Pipeline	6
Figure 2 Dataset Preview	7
Figure 3 Cleaning Summary	8
Figure 4 Quantitative Impact.....	9
Figure 5 Engineered Features.....	10
Figure 6 RDD Conversion.....	11
Figure 7 Top Crime Districts	12
Figure 8 Top Crime Types	12
Figure 9 Crime Distribution by Hour	12
Figure 10 Crime Distribution by Weekday	13
Figure 11 Temporary View	13
Figure 12 Query 1	14
Figure 13 Query 2	14
Figure 14 Query 3	15
Figure 15 Query 4	15
Figure 16 Query 5	15
Figure 17 Descriptive Statistics	16
Figure 18 Silhouette Scores Produced by KMeans Models	17
Figure 19 Baseline vs Final Model Comparison	17
Figure 20 Distribution of Crime Incidents Across Clusters.....	18
Figure 21 Sample Clustering Predictions	18
Figure 22 Cluster Centers	18

Table of Tables

Table 1 Encoding	10
Table 2 Summary Transformation	11
Table 3 Silhouette Scores for k = 2 to 6.....	17

1. Introduction

1.1 Background and Context

Crime analysis plays an important role in public safety and hotspot detection. Identifying spatial and temporal crime patterns can help authorities allocate resources more effectively and improve decision-making processes.

Apache Spark provides a scalable distributed computing framework that supports RDD operations, Spark SQL analytics, and machine learning through MLlib, making it suitable for large-scale crime pattern analysis.

This project, titled Chicago Vanguard: Big Data Crime Pattern Analysis Using Apache Spark, applies Spark-based preprocessing, distributed analytics, and unsupervised machine learning techniques to analyze crime patterns within the Chicago Crime dataset. The project focuses on discovering spatial and temporal crime trends, identifying hotspot locations, and analyzing crime behavior using scalable big data technologies.

1.2 Problem Statement

The Chicago Crime dataset contains millions of crime incident records collected over many years, making manual analysis and traditional single-machine processing inefficient and difficult to scale. The dataset includes diverse attributes such as crime type, location, district, time of occurrence, arrest information, and geographic coordinates.

In addition, raw crime datasets often contain missing values, inconsistent formats, duplicate records, invalid geographic coordinates, and noisy categorical information. Without proper preprocessing and scalable analysis techniques, extracting meaningful crime patterns becomes unreliable and computationally expensive.

The main problem addressed in this project is how to efficiently preprocess, analyze, and model large-scale crime data using Apache Spark in order to identify meaningful spatial and temporal crime patterns and support crime hotspot analysis.

1.3 Objectives

- To preprocess and clean the Chicago Crime dataset using scalable Spark-based operations.
- To apply data reduction and feature engineering techniques to improve analytical efficiency.
- To perform distributed crime analysis using Spark RDD transformations and actions.
- To execute advanced analytical queries using Spark SQL.
- To apply unsupervised machine learning techniques using Spark MLlib for crime pattern clustering.
- To identify spatial and temporal crime hotspots based on geographic, categorical, and time-based features.

1.4 Scope and Limitations

This project focuses on large-scale crime analysis using the publicly available Chicago Crime dataset. The analysis includes preprocessing, distributed RDD operations, SQL-based analytics, and unsupervised machine learning using KMeans clustering within Apache Spark.

The project primarily analyzes spatial and temporal crime behavior using selected features such as geographic coordinates, crime occurrence time, crime frequency, and arrest-related attributes. The implemented machine learning task focuses on clustering crime incidents into meaningful groups rather than performing supervised crime prediction.

Crime behavior is influenced by external social and environmental factors that are not included in the dataset. In addition, KMeans may not fully capture highly complex crime distributions.

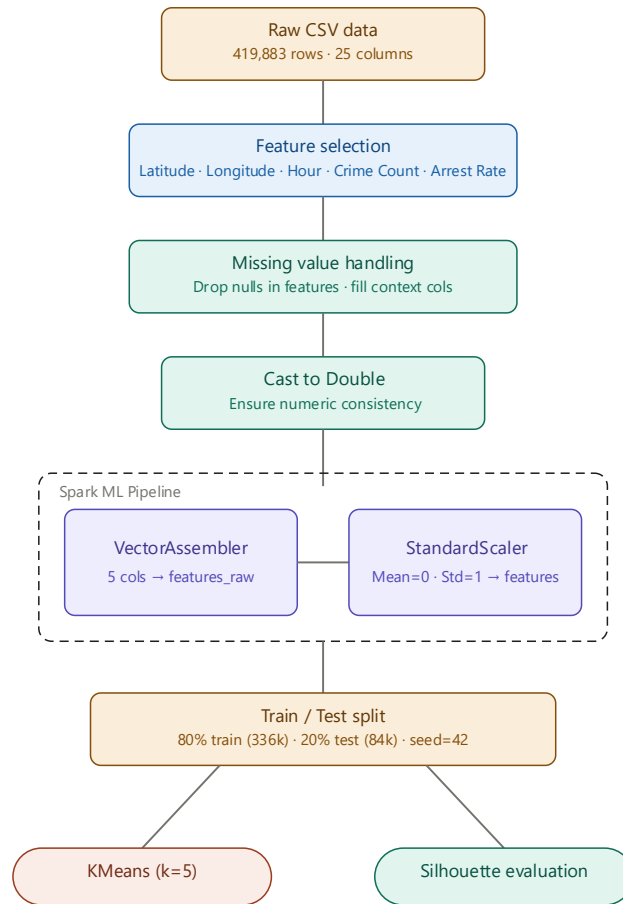


Figure 1 Crime Analysis Pipeline

2. Data Selection and Understanding

2.1 Dataset Description

The dataset used in this project is the Chicago Crime Dataset, publicly available through the City of Chicago Data Portal. The dataset contains reported crime incidents that occurred in Chicago from 2001 to the present and includes millions of records collected over multiple years, making it suitable for large-scale big data analysis using Apache Spark.

The dataset includes information such as crime type, district, community area, arrest status, domestic case indicator, geographic coordinates, and timestamp information. These attributes support both spatial and temporal crime pattern analysis.

2.2 Structure and Schema

The raw dataset originally contained 8,505,782 rows and 22 columns before preprocessing. The dataset combines categorical, numerical, geographic, and temporal attributes related to crime incidents reported across Chicago.

Some of the most important attributes used in this project include:

- Primary Type
- Location Description
- District
- Community Area
- Arrest
- Domestic
- Latitude
- Longitude
- Date

These attributes were selected because they provide valuable information for analyzing crime frequency, temporal behavior, geographic hotspot distribution, and arrest-related patterns.

```
scala> df.show(5)
```

	Primary Type	Location Description	Date	Year	Arrest	Domestic	District	Community Area	Latitude	Longitude	Hour	Weekday	Month	Arrest_Int	Domestic_Int	Is_Weekend	Is_Night	Time_Of_Day
	Crime_Hour_Bin	District_Community_Key	Location_Crime_Count	PrimaryType	ArrestRate	Is_Hotspot	Location	PrimaryType_Index	LocationDesc_Index									
liquor law violation	cta platform	2005-09-14T19:50...	[2005]	true	false	7	69	[41.769185486 -87.625732415]	19	4	9	1	0	0	0	0	0	Evening
liquor law violation	cta platform	2001-11-14T21:00...	[2001]	true	false	1	NULL	[41.868840997 -87.627290382]	71	4	11	1	0	0	0	0	1	Evening
battery	cta platform	2024-06-22T11:20...	[2024]	false	false	11	26	[41.885487535 -87.726422045]	11	7	6	0	0	0	1	0	0	Morning
battery	cta platform	2024-03-20T13:35...	[2024]	false	false	1	32	[41.878574437 -87.633743052]	13	3	3	0	0	0	0	0	0	Afternoon
battery	cta platform	2023-06-21T00:20...	[2023]	false	false	16	11	[41.97008868 -87.762690091]	0	4	6	0	0	0	0	0	1	Night

only showing top 5 rows

Figure 2 Dataset Preview

2.3 Initial Data Quality Observations

Several data quality issues were identified during the initial inspection of the dataset. Missing values were present in location-related columns and geographic coordinates. Some records also contained invalid timestamps, duplicate records, inconsistent text formatting, and outlier geographic coordinates outside the valid Chicago area boundaries.

In addition, categorical attributes contained inconsistencies caused by spacing and capitalization differences, requiring preprocessing operations such as cleaning, standardization, null handling, deduplication, type conversion, and outlier treatment before performing large-scale analytics and machine learning tasks.

Issue	Before	After
Total Rows	8505782	8410907
Rows Removed	N/A	94875
Missing Lat/Lon	94726	0
Missing Location Description	15664	0
Duplicate IDs	0	0
Invalid Coordinates	149	0

Figure 3 Cleaning Summary

2.4 Ethical and Legal Considerations

The Chicago Crime dataset used in this project is publicly available and does not contain personally identifiable information. The analysis focused on aggregate crime patterns for academic purposes only, while maintaining responsible and ethical use of the dataset.

3. Data Preprocessing

3.1 Cleaning

The raw Chicago Crime dataset required preprocessing before performing distributed analytics and machine learning tasks. Several cleaning operations were applied to improve data quality and consistency.

Rows containing completely null values were removed first. For critical attributes such as ID, Date, Primary Type, Latitude, and Longitude, records containing null values were discarded. Non-critical categorical columns such as Location Description, Block, and Description were filled with “UNKNOWN”, while missing numerical attributes were filled using default numerical values.

Duplicate crime incidents were removed based on the ID column. Timestamp validation was performed by converting the Date column into a proper timestamp format and removing invalid records.

Geographic validation was applied to remove coordinates outside the valid Chicago area boundaries. In addition, textual attributes such as Primary Type and Location Description were standardized through trimming and uppercase conversion.

Finally, outlier treatment was applied to Latitude and Longitude using percentile-based capping to reduce the impact of extreme geographic values.

Metric	Before	After
Rows	8415057	3464
Columns	22	4

Figure 4 Quantitative Impact

Quantitative impact of preprocessing and data reduction operations on the dataset.

3.2 Reduction

Data reduction techniques were applied to decrease dataset size and computational complexity while preserving the information required for crime hotspot analysis.

Because the dataset contained more than eight million records, random sampling was applied using Spark's `sample()` function. A 5% random sample was selected without replacement using a fixed seed to ensure reproducibility.

Feature selection was also performed to remove unnecessary attributes that did not contribute directly to hotspot analysis. Important temporal, spatial, and categorical attributes such as Date, Primary Type, District, Community Area, Latitude, Longitude, Arrest, and Domestic were retained.

In addition, aggregation operations were performed to summarize crime incidents across districts, community areas, and crime categories in order to support hotspot identification and improve analytical efficiency.

3.3 Transformation

Several data transformation operations were applied to prepare the dataset for analytics and machine learning tasks using Apache Spark.

Temporal features such as Hour, Weekday, and Month were extracted from the Date column, while Arrest and Domestic were converted into integer representations.

Categorical attributes were transformed using StringIndexer and OneHotEncoder to generate indexed and encoded representations for crime categories and location descriptions.

Conversion	Column(s)	Result
Timestamp → Temporal fields	Date	Hour, Weekday, Month
Boolean → Integer (0/1)	Arrest, Domestic	Arrest_Int, Domestic_Int
Schema cast	Latitude, Longitude, District	Double / Integer

Table 1 Type Conversions

Feature scaling was performed using MinMaxScaler before clustering. Additional engineered features such as Is_Weekend, Is_Night, Time_Of_Day, Crime_Hour_Bin, District_Community_Key, and Is_Hotspot_Location were also generated to improve crime pattern analysis.

3.4 Pipeline Summary

Feature engineering was performed to improve crime pattern analysis by generating new spatial, temporal, and statistical features from the original crime records.

Temporal features such as Hour, Weekday, Month, Is_Weekend, Is_Night, and Time_Of_Day were extracted to support time-based crime analysis. Spatial features including District_Community_Key and Is_Hotspot_Location were generated to improve hotspot identification.

In addition, statistical features such as Location_Crime_Count and PrimaryType_ArrestRate were generated to capture historical crime behavior patterns.

Hour	Is_Weekend	Is_Night	Time_Of_Day	Crime_Hour_Bin	District_Community_Key
11	0	0	Morning	2	7_67
12	1	0	Afternoon	3	6_71
5	0	1	Night	1	9_61
1	0	1	Night	0	10_30
12	1	0	Afternoon	3	12_28

Figure 5 Engineered Features

3.5 Preprocessing Pipeline Summary

The preprocessing workflow integrated cleaning, reduction, transformation, and feature engineering operations into a unified Spark-based pipeline.

The final transformed dataset was exported as a structured CSV file and used throughout the subsequent RDD, SQL, and machine learning phases of the project.

Category	New Columns Added
Temporal fields	Hour, Weekday, Month, Arrest_Int, Domestic_Int
Feature Engineering	Is_Weekend, Is_Night, Time_Of_Day, Crime_Hour_Bin, District_Community_Key, Is_Hotspot_Location
Encoding	PrimaryType_Index, PrimaryType_OHE, LocationDesc_Index, LocationDesc_OHE, PrimaryType_ArrestRate
Scaling output	features_vec, features_scaled

Table 2 Summary Transformation

4. RDD Operations

4.1 RDD Design

After preprocessing, the transformed crime dataset was loaded into Apache Spark and converted into Resilient Distributed Datasets (RDDs) to support distributed processing operations.

Basic preprocessing operations including header removal, record parsing, filtering, and splitting were applied before performing analytical computations.

RDD transformations such as `map()`, `filter()`, `reduceByKey()`, and `sortBy()` were used to process crime records and generate frequency-based and temporal insights.

```
scala> rdd.take(5)
res2: Array[org.apache.spark.sql.Row] = Array([liquor law violation,cta platform,2005-09-14T19:50:45.000+03:00,2005,true,false,7,69,41.769185486,-87.625732415,19,4,0,1,0,0,Evening,4,7,69,2136,0.988204456094364,4,1,18,0,22,0], [liquor law violation,cta platform,2001-11-14T21:00:00.000+03:00,2001,true,false,1,null,41.868040997,-87.627290382,21,4,11,1,0,0,1,Evening,5,null,2136,0.9882044560943644,1,18,0,22,0], [battery,cta platform,2024-06-22T11:20:00.000+03:00,2024,false,false,11,26,41.885487535,-87.726422045,11,7,6,0,0,1,0,Morning,2,11,26,2136,0.21591895428223498,1,1,0,22,0], [battery,cta platform,2024-03-26T13:35:00.000+03:00,2024,false,false,1,32,41.878574487,-87.633743652,13,3,3,0,0,0,Afternoon,3,1,32,2136,0.21591895428223498,1,1,0,22,0], [battery,cta platform,2023-06-21T00:20:00.00...
```

Figure 6 RDD Conversion

4.2 Key Transformations and Actions

RDD-based frequency analysis was performed to identify the most common crime categories and the districts with the highest crime activity levels across Chicago.

```
val crimeCounts = rdd

.map(row => (row.primaryType, 1))

.reduceByKey(_ + _)
```

```
.sortBy(_._2, ascending = false)
```

Crime incidents were grouped by Primary Type and District to calculate crime frequencies and identify hotspot regions within the dataset.

Figures 7 and 8 summarize the districts and crime categories with the highest recorded frequencies.

```
scala> districtCounts.take(10).foreach(println)
(8,28332)
(11,26425)
(6,24762)
(7,23731)
(25,23705)
(4,23685)
(3,21391)
(12,21264)
(9,20249)
(2,20095)
```

Figure 7 Top Crime Districts

```
scala> val crimeTypeCounts = rdd.map(row => (row.getAs[String]("Primary Type"), 1)).reduceByKey(_ + _).sortBy(_._2, false)
crimeTypeCounts: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[29] at sortBy at <console>:23

scala> crimeTypeCounts.take(10).foreach(println)
(theft,88873)
(battery,77191)
(criminal damage,48214)
(narcotics,37310)
(assault,28367)
(other offense,26600)
(burglary,22335)
(motor vehicle theft,21557)
(deceptive practice,18602)
(robbery,15828)
```

Figure 8 Top Crime Types

4.3 Insights

Temporal crime analysis was conducted using RDD transformations to examine crime activity across different hours and weekdays.

Crime records were grouped according to the extracted Hour and Weekday features generated during preprocessing. Aggregation operations were then performed to calculate crime frequencies for each time interval and weekday category.

The analysis revealed noticeable temporal variations in crime activity, with certain hours and days exhibiting higher crime frequencies than others.

```
scala> val hourCounts = rdd.map(row => (row.getAs[String]("Hour").toInt, 1)).reduceByKey(_ + _).sortBy(_._2, false)
hourCounts: org.apache.spark.rdd.RDD[(Int, Int)] = MapPartitionsRDD[104] at sortBy at <console>:23

scala> hourCounts.take(10).foreach(println)
(12,23900)
(0,23827)
(20,23581)
(19,23406)
(18,22889)
(21,22637)
(15,22386)
(22,22124)
(17,21874)
(16,21605)
```

Figure 9 Crime Distribution by Hour

```
scala> val weekdayCounts = rdd.map(row => (row.getAs[String]("Weekday").toInt, 1)).reduceByKey(_ + _).sortBy(_._2, false)
weekdayCounts: org.apache.spark.rdd.RDD[(Int, Int)] = MapPartitionsRDD[111] at sortBy at <console>:23

scala> weekdayCounts.take(10).foreach(println)
(6,63013)
(4,60263)
(7,60210)
(3,59860)
(5,59422)
(2,59289)
(1,57667)
```

Figure 10 Crime Distribution by Weekday

4.4 Summary of RDD Findings

RDD operations successfully identified major spatial and temporal crime distribution patterns across the dataset.

5. SQL Operations

5.1 Temp View Creation

After completing preprocessing and RDD-based analysis, Spark SQL was used to perform structured analytical queries on the transformed crime dataset.

The transformed dataset was loaded into a Spark DataFrame and registered as a temporary SQL view. This enabled the execution of analytical queries involving filtering, grouping, aggregation, ordering, and ranking operations across the dataset.

Spark SQL simplified large-scale analytical operations compared to lower-level RDD transformations.

```
df.createOrReplaceTempView("crimes")
println("Temp view 'crimes' created successfully.")
```

Figure 11 Temporary View

5.2 Key Analytical Queries

Several SQL aggregation queries were executed to analyze crime distribution patterns across districts, crime categories, and temporal attributes.

```
SELECT Primary_Type, COUNT(*) AS Crime_Count
FROM crime_data
GROUP BY Primary_Type
ORDER BY Crime_Count DESC
LIMIT 10;
```

Queries involving GROUP BY, COUNT, ORDER BY, and filtering operations were used to identify districts with the highest crime frequency and the most common crime categories.

The generated query outputs provided structured summaries that supported hotspot analysis and revealed major crime behavior trends across Chicago.

```
=====
Query 1: Number of Crimes per District
=====
```

District	Total_Crimes	Total_Arrests	Arrest_Rate_Pct
8	28422	6131	21.57
11	26397	10657	40.37
6	24843	6059	24.39
4	23709	5072	21.39
25	23602	6055	25.65
7	23500	6459	27.49
3	21597	5190	24.03
12	21251	4516	21.25
9	20271	5470	26.98
2	20034	4937	24.64
18	18987	4278	22.53
19	18886	3789	20.06
5	18461	4465	24.19
10	17921	5506	30.72
15	17756	6748	38.00
1	17584	4620	26.27
14	16149	3085	19.10
16	13998	2545	18.18
22	13723	2700	19.67
24	12838	2849	22.19

```
only showing top 20 rows
```

Figure 12 Query 1

```
=====
Query 2: Most Common Crime Types with Arrest Rate
=====
```

Crime_Type	Total_Crimes	Total_Arrests	Arrest_Rate_Pct	Domestic_Cases
theft	88873	9798	11.02	3158
battery	77191	16667	21.59	40796
criminal damage	48214	3103	6.44	5533
narcotics	37310	37075	99.37	21
assault	28367	5689	20.05	8287
other offense	26600	4622	17.38	9949
burglary	22335	1290	5.78	290
motor vehicle theft	21557	1547	7.18	209
deceptive practice	18602	2314	12.44	300
robbery	15828	1546	9.77	289

Figure 13 Query 2

5.3 SQL Insights

More advanced SQL operations including subqueries, Common Table Expressions (CTEs), and ranking functions were performed to support deeper crime analytics.

Window functions and ordering operations were applied to rank districts and crime categories according to incident frequency. These analytical queries improved the interpretability of crime hotspot patterns and enabled more detailed spatial and temporal comparisons across the dataset.

Figures 14 through 16 present examples of advanced SQL analytical queries and their outputs.

Time_Of_Day	Is_Night	Total_Crimes	Total_Arrests	Arrest_Rate_Pct	Pct_Of_All_Crimes
Afternoon	0	130465	32933	25.24	31.07
Evening	1	86774	26310	30.32	20.67
Morning	0	85243	18406	21.59	20.30
Night	1	70742	14406	20.36	16.85
Evening	0	46659	13188	28.26	11.11

Figure 14 Query 3

Day_Type	Total_Crimes	Total_Arrests	Arrest_Rate_Pct	Avg_Crime_Hour	Distinct_Crime_Types
Weekday	302097	76404	25.29	13.31	31
Weekend	117786	28839	24.48	12.56	32

Figure 15 Query 4

District	Crime_Type	Total_Crimes	Total_Arrests	Arrest_Rate_Pct	Night_Crimes	Weekend_Crimes	Night_Crime_Pct	Weekend_Crime_Pct	district_rank
NULL	assault	1	1	100.00	0	0	0.00	0.00	1
NULL	robbery	1	0	0.00	1	0	100.00	0.00	1
NULL	deceptive practice	1	0	0.00	0	0	0.00	0.00	1
1	theft	7481	1226	16.39	1236	1854	16.52	24.78	1
1	battery	1893	525	27.73	753	553	39.78	29.21	2
1	deceptive practice	1612	322	19.98	286	341	17.74	21.15	3
2	theft	4893	465	11.36	1095	1046	26.75	25.56	1
2	battery	3943	777	19.71	1595	1158	40.45	29.37	2
2	criminal damage	2077	119	5.73	895	597	43.89	28.74	3
3	battery	4739	1804	21.19	2067	1459	43.62	30.79	1
3	theft	3219	319	9.91	925	822	28.74	25.54	2
3	criminal damage	2617	142	5.43	1186	779	45.32	29.77	3
4	battery	4982	930	18.67	2037	1469	40.89	29.49	1
4	theft	3778	338	8.94	1164	929	30.81	24.59	2
4	criminal damage	3158	150	4.78	1492	1017	47.55	32.41	3
5	battery	4881	826	20.24	1678	1227	41.12	30.07	1
5	theft	2609	221	8.47	732	693	28.06	26.56	2
5	criminal damage	2363	130	5.50	1039	722	43.97	30.55	3
6	battery	5192	1090	20.99	2281	1616	43.93	31.12	1
6	theft	4336	669	15.43	1233	1125	28.44	25.95	2
6	criminal damage	2770	182	6.57	1199	829	43.29	29.93	3
7	battery	5846	1047	17.91	2524	1770	43.17	30.28	1
7	theft	3892	218	7.05	922	768	29.82	24.84	2
7	criminal damage	2641	172	6.51	1222	846	46.27	32.03	3
8	theft	5391	659	12.22	1609	1391	29.85	25.80	1
8	battery	4828	1016	21.04	1996	1582	41.34	32.77	2
8	criminal damage	4127	238	5.77	2072	1299	50.21	31.48	3
9	battery	3892	887	22.79	1749	1292	44.94	33.20	1
9	theft	3589	458	12.76	1010	985	28.14	27.44	2
9	criminal damage	2672	196	7.34	1311	845	49.86	31.62	3
10	battery	3995	851	21.30	1761	1373	44.08	34.37	1
10	theft	2582	249	9.64	716	680	27.73	26.34	2
10	narcotics	2341	2324	99.27	862	616	36.82	26.31	3
11	narcotics	6569	6550	99.71	1791	1754	27.26	26.70	3

Figure 16 Query 5

5.4 Summary of SQL Findings

Spark SQL queries provided structured analytical insights into crime distribution, hotspot regions, and temporal behavior patterns.

6. Machine Learning Operations

6.1 Problem Formulation

The machine learning phase of this project focused on identifying hidden crime behavior patterns within the Chicago Crime dataset using unsupervised learning techniques.

Since the dataset does not contain predefined hotspot labels, the problem was formulated as a clustering task using KMeans within Spark MLlib.

The clustering model grouped crime incidents according to spatial, temporal, and contextual similarities using features such as geographic coordinates, crime occurrence time, location crime frequency, and arrest-related attributes.

6.2 Feature Pipeline and Preprocessing

Before training the clustering model, several preprocessing operations were applied to prepare the dataset for machine learning analysis. Important numerical features including Latitude, Longitude, Hour, Location_Crime_Count, and PrimaryType_ArrestRate were selected for clustering.

```
val assembler = new VectorAssembler()

.assembleInputCols(featureCols)

.setOutputCol("features")
```

Rows containing null values in critical feature columns were removed to preserve clustering quality.

A Spark ML Pipeline was constructed using VectorAssembler and StandardScaler to combine and normalize numerical features before KMeans clustering.

summary	Latitude	Longitude	Hour	Location_Crime_Count	PrimaryType_ArrestRate
count	419883	419883	419883	419883	419883
mean	41.842406956030715	-87.67126385690644	13.098744173972273	51370.25636427291	0.25064839491003005
stddev	0.08980122030383618	0.061979081995622555	6.743998705549545	41702.99595999077	0.2832795897673914
min	36.619446395	-91.686565684	0.0	1.0	0.0
max	42.022642635	-87.524529465	23.0	110079.0	1.0

Figure 17 Descriptive Statistics

6.3 Model Training and Evaluation

The transformed dataset was randomly divided into training and testing subsets using

an 80/20 split ratio. Multiple KMeans models were trained using values of k ranging from 2 to 6.

Model evaluation was performed using the Silhouette score to measure cluster compactness and separation quality. The evaluation results showed that k = 5 achieved the highest silhouette score among all tested configurations.

```
scala> (2 to 6).foreach(k=>val model=new KMeans().setK(k).setSeed(42).setFeaturesCol("features").setPredictionCol(
"prediction").fit(trainDF); val preds=model.transform(testDF); val score=new ClusteringEvaluator().setFeaturesCol(
"features").setPredictionCol("prediction").setMetricName("silhouette").evaluate(preds); println("k="+k+" -> silhou
ette="+score))
[Stage 18:>] (0 + 8) [Stage 18:=====] (4 + 4)
(1 + 7) [Stage 21:=====] (3 + 5) [Stage 21:>] (1 + 7)
[Stage 21:=====] (0 + 8) [Stage 21:=====] (3 + 5) [Stage 21:>] (1 + 7)
26/05/09 20:23:07 WARN InstanceBuilder: Failed to load implementation from: dev.ludovic.netlib.blas.JNIBLAS
[Stage 69:>] (1 + 7) [Stage 69:=====] (0 + 8) [Stage 69:=====] (3 + 5)
[Stage 69:=====] (7 + 1) [Stage 69:=====] (0 + 8) [Stage 69:>] (3 + 5)
[Stage 72:=====] (1 + 7) [Stage 72:=====] (0 + 8) [Stage 72:>] (3 + 5)
[Stage 74:>] (5 + 3) (0 + 8) [Stage 74:=====] (0 + 8) [Stage 74:>] (3 + 5)
K=2 -> silhouette=0.33041409986853026 (0 + 8) [Stage 77:=====] (0 + 8) [Stage 77:>] (3 + 5)
[Stage 77:>] (1 + 7) (0 + 8) [Stage 80:=====] (0 + 8) [Stage 80:>] (3 + 5)
[Stage 80:>] (1 + 7) [Stage 80:=====] (0 + 8) [Stage 80:>] (3 + 5)
(0 + 8) [Stage 128:=====] [Stage 128:>] (2 + 6)
(1 + 7) [Stage 131:=====] [Stage 131:>] (7 + 1)
(1 + 7) [Stage 133:=====] [Stage 133:>] (7 + 1)
[Stage 136:=====] (1 + 7) K=3 -> silhouette=0.3122989573043786
[Stage 139:>] (5 + 3) (0 + 8) [Stage 139:=====] (4 + 4)
[Stage 139:>] (1 + 7) [Stage 139:=====] [Stage 190:=====] (4 + 4)
[Stage 192:=====] (1 + 7) [Stage 192:=====] (4 + 4)
K=4 -> silhouette=0.32172990345900143 (1 + 7) [Stage 195:=====] (5 + 3)
[Stage 195:=====] (1 + 7) [Stage 195:=====] (0 + 8) [Stage 198:=====] (1 + 7) [Stage 198:>] (2 + 6)
[Stage 246:=====] (2 + 6) (1 + 7) [Stage 246:=====] (4 + 4)
[Stage 249:=====] K=5 -> silhouette=0.34665664710701505 (1 + 7) [Stage 254:=====] (4 + 4)
[Stage 254:=====] (4 + 4) (0 + 8) [Stage 257:=====] (7 + 1)
[Stage 257:>] (1 + 7) [Stage 257:=====] [Stage 305:=====] (4 + 4)
[Stage 310:=====] (1 + 7) K=6 -> silhouette=0.32347392942795705
```

Figure 18 Silhouette Scores Produced by KMeans Models

k	Silhouette Score
2	0.3304
3	0.3123
4	0.3217
5	0.3467
6	0.3235

Table 3 Silhouette Scores for k = 2 to 6

A baseline KMeans model using k = 2 was compared against the final selected model using k = 5. The final model achieved improved cluster separation and more balanced crime pattern grouping across the dataset.

```
scala> println("Baseline K=2 Silhouette score: "+baselineScore); println("Final K=5
Silhouette score: "+finalSilhouette)
Baseline K=2 Silhouette score: 0.33041409986853026
Final K=5 Silhouette score: 0.34665664710701505
```

Figure 19 Baseline vs Final Model Comparison

6.4 Cluster Analysis and Interpretation

The final KMeans model generated multiple crime clusters representing different spatial and temporal crime behavior patterns across Chicago.

Cluster distribution analysis showed that the dataset was partitioned into reasonably balanced groups without producing empty clusters. Figure 20 illustrates the distribution of crime incidents across the generated clusters.

```
scala> finalPredictions.groupBy("prediction").count().orderBy("prediction").show(false)
[Stage 431:=====>                                     (1 + 7)
+-----+-----+
---+
|prediction|count|
+-----+-----+
|0         |11708|
|1         |23235|
|2         |14644|
|3         |20069|
|4         |14088|
+-----+-----+
```

Figure 20 Distribution of Crime Incidents Across Clusters

In addition, cluster center analysis revealed noticeable differences between hotspot regions and lower-risk areas based on the selected numerical features.

```
scala> finalPredictions.select("Primary Type", "District", "Time_Of_Day", "Is_Night", "Is_Weekend", "prediction").show(20, false)
+-----+-----+-----+-----+-----+-----+
|Primary Type|District|Time_Of_Day|Is_Night|Is_Weekend|prediction|
+-----+-----+-----+-----+-----+-----+
|motor vehicle theft|4|Morning|0|0|1|
|criminal damage|5|Night|1|0|1|
|deceptive practice|4|Morning|0|0|1|
|criminal damage|4|Night|1|0|1|
|motor vehicle theft|4|Morning|0|0|1|
|criminal damage|4|Evening|0|0|1|
|battery|4|Night|1|1|1|
|assault|4|Afternoon|0|0|1|
|theft|4|Night|1|0|1|
|deceptive practice|4|Morning|0|0|1|
|other offense|4|Morning|0|0|1|
|battery|4|Afternoon|0|0|1|
|deceptive practice|4|Evening|1|0|1|
|assault|5|Afternoon|0|1|1|
|battery|5|Afternoon|0|0|1|
|narcotics|5|Morning|0|1|0|
|criminal damage|4|Night|1|0|1|
|theft|4|Afternoon|0|0|1|
|theft|5|Afternoon|0|1|1|
|theft|5|Morning|0|0|1|
+-----+-----+-----+-----+-----+-----+
only showing top 20 rows
```

Figure 21 Sample Clustering Predictions

```
scala> finalModel.clusterCenters.zipWithIndex.foreach{case(c,i)=>println("Cluster "
+i+": "+c)}
Cluster 0: [-0.027324868843108446,-0.1252959266893183,0.3043833417610703,0.03532294
7814623064,2.332530407324642]
Cluster 1: [-1.0580630950977932,0.8894901464458264,0.06883190830219538,0.1183800997
7705885,-0.3571827587098141]
Cluster 2: [0.5913470817185492,-0.6032777032845595,0.5898415760681096,0.96240497486
13429,-0.41221781434686205]
Cluster 3: [0.5621345806664283,-0.2937103428988075,0.3598650543082785,-0.9493636527
893583,-0.40031061009434754]
Cluster 4: [0.3748042316388755,-0.33583479453454956,-1.4645456384936013,0.130507761
6113906,-0.35624255836608243]
scala> |
```

Figure 22 Cluster Centers

The inclusion of engineered features such as `Location_Crime_Count` and `PrimaryType_ArrestRate` improved the model's ability to capture contextual crime behavior patterns beyond basic geographic clustering.

6.5 Limitations and Future Improvements

Despite producing meaningful clustering results, several limitations remain within the current machine learning implementation. Crime behavior is influenced by external social and environmental factors that are not included in the selected feature set.

In addition, KMeans may not fully represent highly complex real-world crime distributions.

Future work could explore additional feature engineering techniques and alternative clustering algorithms such as DBSCAN or Gaussian Mixture Models.

7. Conclusion

This project demonstrated the use of Apache Spark for large-scale crime data analysis using preprocessing, distributed analytics, Spark SQL, and machine learning techniques.

RDD operations and Spark SQL queries provided insights into crime frequency, hotspot regions, and temporal crime distribution trends. In addition, the machine learning phase successfully applied KMeans clustering using Spark MLlib to group crime incidents into meaningful behavioral clusters.

The results showed that distributed big data technologies can efficiently process large crime datasets while supporting scalable analytical and machine learning workflows.

8. References

- [1] Apache Spark Documentation. <https://spark.apache.org/docs/latest/>
- [2] Apache Spark MLlib Guide. <https://spark.apache.org/mllib/>
- [3] City of Chicago Data Portal – Crimes 2001 to Present. <https://data.cityofchicago.org/>
- [4] Jain, A. K. "Data Clustering: 50 Years Beyond K-Means." Pattern Recognition Letters, 2010.

[5] Rousseeuw, P. J. "Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis." *Journal of Computational and Applied Mathematics*, 1987.

[6] Zaharia, M., et al. "Apache Spark: A Unified Engine for Big Data Processing." *Communications of the ACM*, 2016.